

Оракул через посаженный инвариант

Метод исследования закрытой системы без доступа к её внутренностям.

Домен-независимая формулировка. Первое приложение — self-ADINT, но оно не единственное.

Ход

Найти величину, которую система не может испортить, потому что эта величина нужна ей самой. Положить в неё собственную метку. Прочитать выход.

Ключевой сдвиг: не «наблюдать за системой», а **контролировать её вход**.

Без контроля входа любое наблюдение даёт корреляции. С контролем — причинность, потому что источник факта известен точно.

Условия применимости

Метод работает только если выполнены все четыре:

1. **Есть канал записи.** Хоть какое-то поле, значение или поведение, которое ты задаёшь и которое система принимает.
2. **Есть наблюдаемый выход.** Не обязательно полный — достаточно бита.

3. **Существует инвариант.** Величина, проходящая конвейер без деградации.

Обычно это то, вокруг чего построена экономика системы: ключи связывания, идентификаторы, всё, за порчу чего кто-то платит.

4. **Метка глобально уникальна.** Иначе возврат — совпадение, а не доказательство.

Если инварианта нет — метод не применим, остаётся статистика.

Как ищется инвариант

Не угадыванием. Инвариант читается из функции потерь конвейера:

что сохраняется точно — то монетизируется.

Практический порядок:

- посмотреть, что система продаёт;
- то, без чего продукт разваливается, и есть инвариант;
- проверить на данных: прогнать заведомо разные величины, посмотреть, какие доезжают побитово, какие огрубляются, какие схлопываются в категорию.

Конкурирующее объяснение, которое обязано быть исключено: **инерция кода**.

Поле может доезжать нетронутым не потому, что оно ценно, а потому что переписать древний SDK дороже, чем оставить. Отличается по внешним признакам (есть ли рынок на это поле), а не по самим данным.

Что вытаскивается, по возрастанию силы

1. **Связность.** Метка вернулась → путь существует. Один бит, но не вероятностный.
2. **Топология.** Разная метка на каждый источник → таблица рёбер.
Не «утекло», а «утекло отсюда туда».
3. **Время.** Момент посадки известен точно, момент возврата наблюдаем.
Разница — реальная задержка конвейера. Момент, когда метка перестаёт приходить, — фактический срок хранения. Измеренный, а не декларированный.
4. **Провенанс.** Без контролируемого входа не отличить «система это вывела» от «системе это сказали». Посаженный факт снимает неоднозначность полностью.
5. **Модель вывода.** Посадить факт ложный, но правдоподобный, и смотреть, что вокруг него нарастает. Вход известен точно, значит выход — чистый снимок того, **как система думает**, а не что она знает.

Трёхзначный исход

Оракул почти всегда односторонний. Возврат метки доказывает путь.

Отсутствие возврата не доказывает ничего — неотличимо от отсутствия okazji, таймаута, сэмплирования.

YES	— метка вернулась
UNKNOWN	— по умолчанию
NO	— к отрицаний подряд

k вычисляется из наблюдаемой базовой частоты оказий, не берётся константой.

UNKNOWN не расщепляется и не рекурсируется — переставляется в хвост очереди.

Это не украшение. Любой алгоритм поиска, строящий рекурсию на отрицаниях, на шумном одностороннем оракуле разъезжается.

Выбор алгоритма опроса

Пропускная способность оракула — главный дефицит. Не деньги, не вычисления: число оказий в единицу времени.

Тест монотонный, интерференции нет → group testing (Дорфман / binary splitting).

Один тест на пул. Порядок экономии: при n кандидатов и малом числе попаданий — логарифмический вместо линейного.

Тест немонотонный, элементы интерферируют → ddmin.

Его машинерия (тест дополнений, повышение гранулярности при провале обеих половин) существует именно для этого случая и в монотонном вырождается в обычное бинарное расщепление, платя за то, чего не происходит.

Различающий вопрос один: **может ли добавление элемента превратить YES в NO?**

Нет → group testing. Да → ddmin.

Отдельно считать длину цикла. ddmin — $O(n^2)$ в худшем случае. Если один тест занимает дни (нужно ждать переобучения системы), реалистичный n — единицы.

Порядок фаз

Инъекция необратимо загрязняет baseline. Второго чистого профиля не будет.

1. Пассивное чтение. Что доезжает даром. Замер базовой частоты okazji → отсюда к.
2. Карта инвариантов. Что сохраняется точно, что огрубляется, что схлопывается.
3. Посадка меток. Уникальная на каждый источник. Только теперь.
4. Опрос. Group testing или ddmin по результату шага 2.
5. Ложный факт. Чтение модели вывода. Точка невозврата для baseline.

Границы

- Инвариантность условная, а не абсолютная. Держится ровно внутри контура, который заинтересован её держать. Соль на стороне системы, k-анонимизация, смена схемы — метка мертва. «Переживёт любые пертурбации» — неверно.
- Писать можно только в pass-through поля и в поведение. Вычисленные системой величины напрямую не редактируются — только косвенно, через вход, с задержкой.
- Метод даёт факты о путях, а не о намерениях.

Родственники

Copyright trap: фиктивные улицы у картографов, mountweazels в словарях, canary token в безопасности. Разница в том, что там метка

ловит копирование,
а здесь — служит зондом для реконструкции конвейера.

Ближайший интеллектуальный предок по существу — не они,
а сам ход
«верификация, направленная наружу, а не в self-critique».
Здесь он применён к системе, которой ты не владеешь.